

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: MLA03

COURSE NAME: Reinforcement learning for Environment and Agent

COURSE OUTCOME COVERED

CO3: Implement policy-based reinforcement learning methods to develop efficient solutions for complex environments. (BL2, BL3, BL6)

Assessment Tool Weightage: 30%

Dr G. A. SENTHIL

QUESTIONS: 10

Total Marks: 50

Annexure A

CODING CHALLENGE

Duration: 2hrs

1. Predict the Reinforcement Learning (RL) framework by describing the agent–environment interaction, emphasizing the importance of states, actions, and rewards, and discuss how cumulative reward influences decision-making. **(5 marks)**

A. Problem Statement

Reinforcement Learning (RL) is a machine learning technique in which an agent learns by interacting with an environment. The agent observes the current state, performs an action, receives a reward, and moves to the next state. The objective is to learn the best actions that maximize the total cumulative reward.

Objectives

- To understand the basic concept of Reinforcement Learning.
- To identify states, actions, and rewards.
- To understand the interaction between an agent and its environment.
- To learn how cumulative rewards help in selecting better actions.
- To implement a simple RL example using Python.

Eligibility

- Basic knowledge of Python programming.
- Basic understanding of Machine Learning concepts.
- Knowledge of states, actions, and rewards is helpful.
- Suitable for students learning AI/ML and Reinforcement Learning.

Challenge Rules

1. Understand the agent–environment interaction.
2. Identify the state of the environment.

3. Select an appropriate action.
4. Receive a reward or penalty based on the action.
5. Use cumulative rewards to learn the best decision.
6. The agent should try to maximize the total reward over time.

RL Framework

Agent $\rightarrow$ Action $\rightarrow$ Environment $\rightarrow$ Reward + New State $\rightarrow$ Agent

- Agent: Learner or decision-maker.
- Environment: The world in which the agent operates.
- State: Current situation of the environment.
- Action: Decision taken by the agent.
- Reward: Feedback received after taking an action.
- Cumulative Reward: Total reward collected over multiple steps.

Importance of Cumulative Reward

Python Program

```
# Simple Reinforcement Learning Example
```

```
# States
```

```
states = ["Start", "Middle", "Goal"]
```

```
# Actions and rewards
```

```
actions = {
```

```
    "Start": {"Move Right": 1, "Stay": -1},
```

```
    "Middle": {"Move Right": 10, "Stay": -1}
```

```
}
```

```
state = "Start"
```

```
total_reward = 0
```

```
print("Reinforcement Learning Demonstration")
```

```
print("-----")
```

```
while state != "Goal":
```

```
# Select action with highest reward

action = max(actions[state], key=actions[state].get)

# Get reward

reward = actions[state][action]

# Display information

print("State :", state)

print("Action :", action)

print("Reward :", reward)

# Update cumulative reward

total_reward += reward

# Move to next state

if state == "Start":

    state = "Middle"

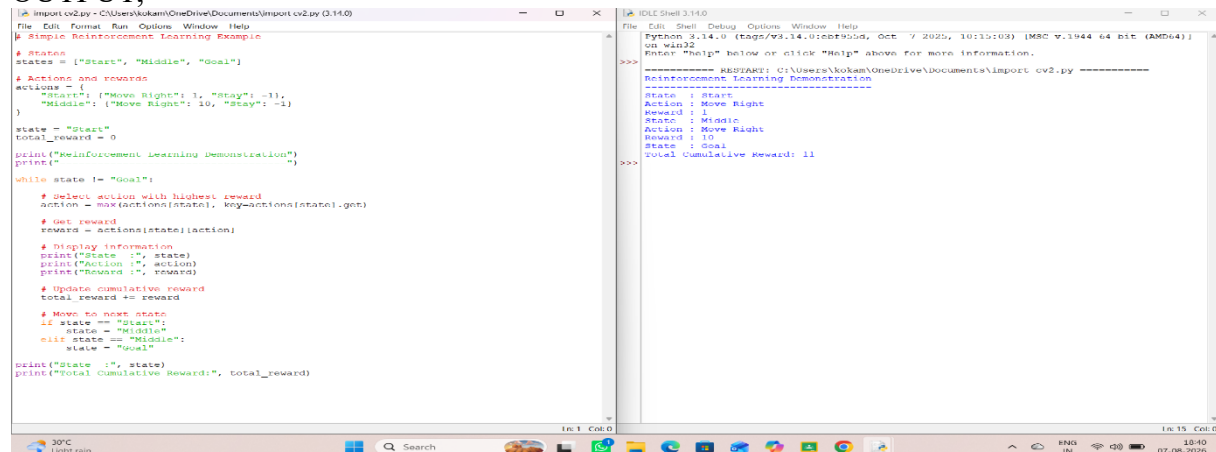
elif state == "Middle":
```

```
state = "Goal"
```

```
print("State :", state)
```

```
print("Total Cumulative Reward:", total_reward)
```

OUTPUT:



```
[*] Import cv2.py - C:\Users\kokam\OneDrive\Documents\Import cv2.py (3.14.0)
File Edit Format Run Options Window Help
# Simple Reinforcement Learning Example
# States
states = ("Start", "Middle", "Goal")
# Actions and rewards
actions = {
    "Start": ["Move Right": 1, "Stay": -1],
    "Middle": ["Move Right": 10, "Stay": -1]
}
state = "Start"
total_reward = 0
print("Reinforcement Learning Demonstration")
print("-----")
while state != "Goal":
    # Select action with highest reward
    action = max(actions[state], key=actions[state].get)
    # Get reward
    reward = actions[state][action]
    # Display information
    print("State :", state)
    print("Action :", action)
    print("Reward :", reward)
    # Update cumulative reward
    total_reward += reward
    # Move to next state
    if state == "Start":
        state = "Middle"
    elif state == "Middle":
        state = "Goal"
    else:
        state = "Goal"
print("State :", state)
print("Total Cumulative Reward:", total_reward)

Python 3.14.0 (tags/v3.14.0:abf955d, Oct 7 2025, 10:15:09) [MSC v.1944 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:\Users\kokam\OneDrive\Documents\Import cv2.py =====
>>>
Reinforcement Learning Demonstration
-----
State : Start
Action : Move Right
Reward : 1
State : Middle
Action : Move Right
Reward : 10
State : Goal
Total Cumulative Reward: 11
>>>
```

2. Estimate how decision-making problems can be modeled as a Markov Decision Process (MDP), detailing the components of action space, transition probability, reward function, and discount factor.

A.Problem Statement

A **Markov Decision Process (MDP)** is a mathematical model used to represent decision-making problems where an agent must choose actions in different states to achieve the best possible result.

An MDP is represented as:

MDP = (S, A, P, R, γ)

Add More

246-2-46-1-2-3-41234-1-2-3-5-4-5

0,0

Components of MDP

1. State Space (S)

Represents all possible situations of the environment.

Example: A robot's current position.

2. Action Space (A)

Represents the possible actions available to the agent.

Example: Move Left, Move Right, Move Forward.

3. Transition Probability (P)

Defines the probability of moving from one state to another after taking an action.

Example: After moving right, there may be a 90% chance of reaching the next position.

4. Reward Function (R)

Gives a reward or penalty for taking an action.

Example: +10 for reaching the goal and -1 for making an unwanted move.

5. Discount Factor (γ)

Determines how much future rewards are considered. Its value is usually between 0 and 1.

- γ close to 0 $\rightarrow$ focuses on immediate rewards.
- γ close to 1 $\rightarrow$ considers future rewards more strongly.

Simple Example

Consider a robot moving toward a destination:

Component	Example
State	Robot's current position
Action	Left, Right, Forward
Transition Probability	Probability of reaching the next position
Reward	+10 for reaching destination, -1 otherwise
Discount Factor	0.9

Python Program

Simple Markov Decision Process Example

```
states = ["Start", "Middle", "Goal"]
```

```
actions = ["Move Right", "Stay"]
```

```
# Transition probabilities
```

```
transition_probability = {  
    ("Start", "Move Right"): {"Middle": 1.0},  
    ("Middle", "Move Right"): {"Goal": 1.0}  
}
```

```
# Reward function
```

```
rewards = {  
    ("Start", "Move Right"): 1,  
    ("Middle", "Move Right"): 10  
}
```

```
# Discount factor
```

```
gamma = 0.9
```

```
state = "Start"
```

```
total_reward = 0
```

```

print("Markov Decision Process")
print("-----")

while state != "Goal":

    action = "Move Right"

    reward = rewards[(state, action)]
    probability = transition_probability[(state, action)]

    next_state = max(probability, key=probability.get)

    print("State:", state)
    print("Action:", action)
    print("Transition Probability:", probability[next_state])
    print("Reward:", reward)
    print("Next State:", next_state)
    print()

    total_reward += reward
    state = next_state

print("Final State:", state)
print("Total Reward:", total_reward)
print("Discount Factor:", gamma)

```

OUTPUT:

The screenshot shows a Python IDE with two panes. The left pane contains the source code for a Markov Decision Process simulation. The right pane shows the output of the program, which includes the state, action, transition probability, reward, and next state at each step, along with the final state, total reward, and discount factor.

```

# Import cv2.py - C:\Users\koka\OneDrive\Documents\import cv2.py 3.14.0
File Edit Format Run Options Window Help
# Simple Markov Decision Process Example
states = ("Start", "Middle", "Goal")
actions = ("Move Right", "Stop")

# Transition probabilities
transition_probability = {
    ("Start", "Move Right"): ("Middle": 1.0),
    ("Middle", "Move Right"): ("Goal": 1.0)
}

# Reward function
rewards = {
    ("Start", "Move Right"): 1,
    ("Middle", "Move Right"): 10
}

# Discount Factor
gamma = 0.9

state = "Start"
total_reward = 0
print("Markov Decision Process")
print("-----")

while state != "Goal":

    action = "Move Right"

    reward = rewards[(state, action)]
    probability = transition_probability[(state, action)]
    next_state = max(probability, key=probability.get)

    print("State:", state)
    print("Action:", action)
    print("Transition Probability:", probability[next_state])
    print("Reward:", reward)
    print("Next State:", next_state)
    print()

    total_reward += reward
    state = next_state

print("Final State:", state)

```

```

===== REPORT: C:\Users\koka\OneDrive\Documents\import cv2.py =====
Markov Decision Process
State: Start
Action: Move Right
Transition Probability: 1.0
Reward: 1
Next State: Middle
State: Middle
Action: Move Right
Transition Probability: 1.0
Reward: 10
Next State: Goal
Final State: Goal
Total Reward: 11
Discount Factor: 0.9

```

(5 marks)

- Discuss the role of policy and value functions in RL, and examine how state-value and action-value functions contribute to evaluating long-term benefits of actions using the Bellman Equation.

A. Problem Statement

In Reinforcement Learning, a **policy** tells the agent what action to take in a particular state. **Value functions** help the agent evaluate how good a state or action is based on the expected future rewards.

1. Policy Function

A **policy** (π) defines the agent's decision-making strategy.

Example:

If the robot is at the starting position:

- Move Right $\rightarrow$ 80% probability
- Move Left $\rightarrow$ 20% probability

The policy helps the agent choose suitable actions.

2. State-Value Function

The **state-value function** $V(s)$ measures the expected long-term reward from a particular state when following a policy.

$|x|$

$$|-3|=3 \quad \left| -3 \right| = 3 \quad |-3|=3$$

$$|x| = -x$$

Distance from 0

-6-5-4-3-2-10123456

Drag the point left or right

It answers:

"How good is this state for the agent?"

3. Action-Value Function

The **action-value function** $Q(s,a)$ measures the expected long-term reward when the agent takes a particular action in a particular state.

$$Q(s,a) = E[G_t | S_t=s, A_t=a]$$

It answers:

"How good is this action in this state?"

4. Bellman Equation

The Bellman Equation breaks the value of a decision into:

Immediate Reward + Discounted Future Value

For state value:

$$V(s) = R + \gamma V(s')$$

where:

- $V(s)$ = current state value
- R = immediate reward
- γ = discount factor
- $V(s')$ = value of the next state

For action value:

$$Q(s,a) = R + \gamma Q(s',a')$$

Importance

- **Policy** → decides what action to take.
- $V(s)$ → evaluates the usefulness of a state.
- $Q(s,a)$ → evaluates the usefulness of an action.

- **Bellman Equation** → connects current rewards with future rewards.
- These functions help the agent make decisions that maximize **long-term benefits**.

Python Program

Policy and Value Function Example in Reinforcement Learning

```
states = ["Start", "Middle", "Goal"]
```

Rewards

```
rewards = {
```

```
    "Start": 1,
```

```
    "Middle": 10,
```

```
    "Goal": 0
```

```
}
```

Discount factor

```
gamma = 0.9
```

Initial state values

```
V = {
```

```
    "Start": 0,
```

```
    "Middle": 0,
```

```
    "Goal": 0
```

```
}
```

```
# Policy
```

```
policy = {
```

```
    "Start": "Move Right",
```

```
    "Middle": "Move Right"
```

```
}
```

```
# Bellman Equation
```

```
V["Middle"] = rewards["Middle"] + gamma * V["Goal"]
```

```
V["Start"] = rewards["Start"] + gamma * V["Middle"]
```

```
print("Policy:")
```

```
for state, action in policy.items():
```

```
    print(state, "->", action)
```

```
print("\nState Values:")
```

```
for state, value in V.items():
```

```
    print(state, "=", value)
```

```
# Action-value example
```

```
Q_Start_Right = rewards["Start"] + gamma * V["Middle"]
```

```
print("\nAction Value:")
```

```
print("Q(Start, Move Right) =", Q_Start_Right)
```

Output:

```
# Import cv2.py - C:\Users\kokam\OneDrive\Documents\import cv2.py (3.14.0)
# Policy and Value Function Example in Reinforcement Learning
states = ["Start", "Middle", "Goal"]

# Rewards
rewards = {
    "Start": 1,
    "Middle": 10,
    "Goal": 0
}

# Discount factor
gamma = 0.9

# Initial state values
V = {
    "Start": 0,
    "Middle": 0,
    "Goal": 0
}

# Policy
policy = {
    "Start": "Move Right",
    "Middle": "Move Right"
}

# Bellman Equation
V["Middle"] = rewards["Middle"] + gamma * V["Goal"]
V["Start"] = rewards["Start"] + gamma * V["Middle"]

print("Policy:")
for state, action in policy.items():
    print(state, "->", action)

print("\nState Values:")
for state, value in V.items():
    print(state, "=", value)

# Action-value example
Q_Start_Right = rewards["Start"] + gamma * V["Middle"]

print("\nAction Value:")
print("Q(Start, Move Right) =", Q_Start_Right)
```

Python 3.14.0 (tags/v3.14.0:ebf555d, Oct 7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

```
>>> ===== RESTART: C:\Users\kokam\OneDrive\Documents\import cv2.py =====
Policy:
Start -> Move Right
Middle -> Move Right

State Values:
Start = 10.0
Middle = 10.0
Goal = 0

Action Value:
Q(Start, Move Right) = 10.0
>>>
```

(5 marks)

4. Justify the exploration–exploitation trade-off in RL and compare strategies such as ϵ -greedy and softmax approaches in balancing learning efficiency. (5 marks)

A. Problem Statement

In Reinforcement Learning, the agent faces a choice between exploration and exploitation.

- Exploration: Trying new actions to discover better rewards.
- Exploitation: Choosing the action that is already known to give a good reward.

A good RL algorithm must balance both to learn efficiently.

1. ϵ -Greedy Strategy

In ϵ -greedy, the agent:

- Chooses a random action with probability $\epsilon \rightarrow$ Exploration.
- Chooses the best-known action with probability $1 - \epsilon \rightarrow$ Exploitation.

For example, if $\epsilon = 0.1$, the agent explores 10% of the time and exploits 90% of the time.

2. Softmax Strategy

Softmax selects actions according to their estimated values.

- Higher-value actions have a higher probability of selection.
- Lower-value actions still have a small chance of being selected.
- A temperature parameter controls the amount of exploration.

Comparison

Feature	ϵ -Greedy	Softmax
Exploration	Random	Value-based
Exploitation	Best action	Higher-value actions preferred
Control	ϵ	Temperature
Simplicity	Very simple	More flexible
Decision	All non-best actions treated similarly	Actions get different probabilities

Why is the Trade-off Important?

Too much exploration wastes time trying poor actions. Too much exploitation may cause the agent to miss a better action. Therefore, balancing both helps the agent learn efficiently and achieve higher long-term rewards.

Python Program


```
import random
```

```
import math
```

```
# Action values
```

```
actions = ["Left", "Right", "Forward"]
```

```
Q = {
```

```
    "Left": 2,
```

```
    "Right": 8,
```

```
    "Forward": 5
```

```
}
```

```
# -----
```

```
# Epsilon-Greedy
```

```
# -----
```

```
epsilon = 0.2
```

```
if random.random() < epsilon:
```

```
epsilon_action = random.choice(actions)
```

```
print("Epsilon-Greedy: Exploring")
```

```
else:
```

```
epsilon_action = max(Q, key=Q.get)
```

```
print("Epsilon-Greedy: Exploiting")
```

```
print("Selected Action:", epsilon_action)
```

```
# -----
```

```
# Softmax
```

```
# -----
```

```
temperature = 1.0
```

```
values = [
```

```
    math.exp(Q[action] / temperature)
```

```
    for action in actions
```

```
]
```

```
total = sum(values)
```

```
probabilities = [
```

```
    value / total
```

```
    for value in values
```

```
]
```

```
softmax_action = random.choices(
```

```
    actions,
```

```
    weights=probabilities,
```

```
    k=1
```

```
)[0]
```

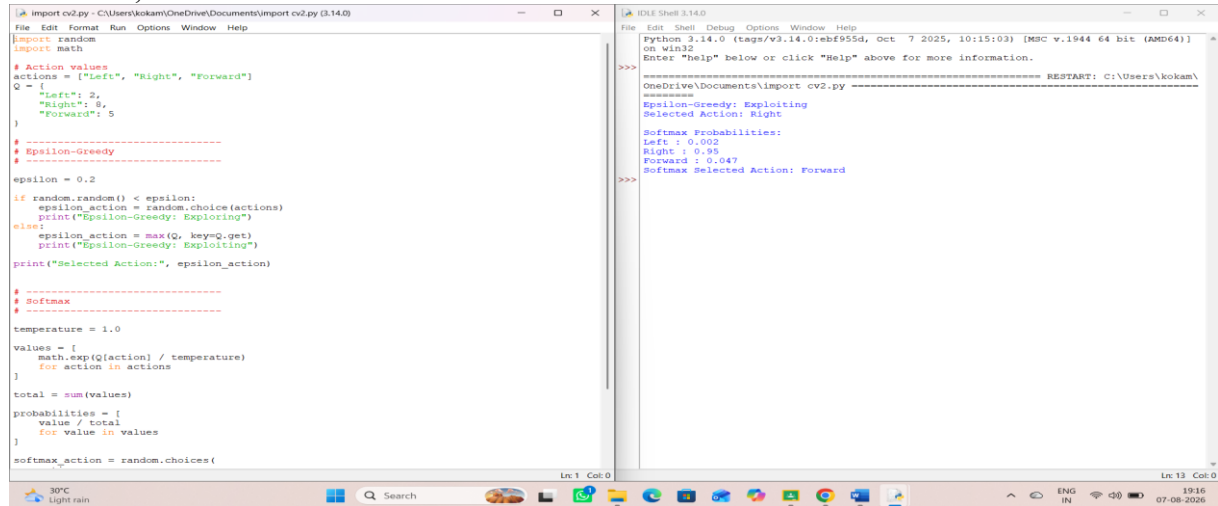
```
print("\nSoftmax Probabilities:")
```

```
for action, probability in zip(actions, probabilities):
```

```
print(action, ":", round(probability, 3))
```

```
print("Softmax Selected Action:", softmax_action)
```

OUTPUT:



```
import cv2.py - C:\Users\kokam\OneDrive\Documents\import cv2.py (3.14.0)
File Edit Format Run Options Window Help

# Action values
actions = ["Left", "Right", "Forward"]
Q = {
    "Left": 2,
    "Right": 8,
    "Forward": 5
}

# -----
# Epsilon-Greedy
# -----

epsilon = 0.2

if random.random() < epsilon:
    epsilon_action = random.choice(actions)
    print("Epsilon-Greedy: Exploring")
else:
    epsilon_action = max(Q, key=Q.get)
    print("Epsilon-Greedy: Exploiting")
print("Selected Action:", epsilon_action)

# -----
# Softmax
# -----

temperature = 1.0
values = [
    math.exp(Q[action]) / temperature
    for action in actions
]
total = sum(values)
probabilities = [
    value / total
    for value in values
]
softmax_action = random.choices(

Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.

>>> ===== RESTART: C:\Users\kokam\
OneDrive\Documents\import cv2.py =====
Epsilon-Greedy: Exploiting
Selected Action: Right

Softmax Probabilities:
Left : 0.002
Right : 0.98
Forward : 0.047
Softmax Selected Action: Forward

>>>
```

5. Explain how reward shaping can accelerate learning in RL and illustrate its impact on preventing suboptimal policies. **(5 marks)**

A.Problem Statement

Reward shaping is a technique of providing additional rewards or penalties to an RL agent to guide it toward the desired behavior. It helps the agent learn faster and avoid choosing suboptimal policies.

How Reward Shaping Helps

- Gives the agent extra feedback during learning.

- Helps the agent understand which actions are useful.
- Reduces the time needed to reach the goal.
- Encourages good actions and discourages bad actions.
- Helps prevent the agent from getting stuck in a suboptimal policy.

Example

Consider a robot moving toward a goal:

Action	Reward
--------	--------

Move toward goal	+5
------------------	----

Move away from goal	-2
---------------------	----

Reach goal	+20
------------	-----

Hit obstacle	-10
--------------	-----

Without reward shaping, the agent may receive a reward only after reaching the goal, making learning slow.

With reward shaping, the agent receives small positive rewards for moving closer to the goal, so it can learn the correct path faster.

Impact on Suboptimal Policies

A suboptimal policy is a strategy that gives a lower long-term reward than the best possible strategy.

Reward shaping guides the agent toward useful actions and makes undesirable actions less attractive. Therefore, it can reduce the chance of the agent repeatedly following a poor strategy.

Python Program

```
# Reward Shaping in Reinforcement Learning
```

```
states = ["Start", "Near Goal", "Goal"]
```

```
# Rewards after reward shaping
```

```
rewards = {
```

```
    ("Start", "Move Forward"): 5,
```

```
    ("Start", "Move Backward"): -2,
```

```
    ("Near Goal", "Move Forward"): 20,
```

```
    ("Near Goal", "Move Backward"): -2
```

```
}
```

```
state = "Start"
```

```
total_reward = 0
```

```
print("Reward Shaping Demonstration")
```

```
print("-----")
```

```
while state != "Goal":
```

```
    # Select the action with the highest reward
```

```
    available_actions = [
```

```
        action for (s, action) in rewards if s == state
```

```
    ]
```

```
    best_action = max(
```

```
        available_actions,
```

```
        key=lambda action: rewards[(state, action)]
```

```
    )
```

```
    reward = rewards[(state, best_action)]
```

```
print("State :", state)
```

```
print("Action:", best_action)
```

```
print("Reward:", reward)
```

```
total_reward += reward
```

```
# Move to next state
```

```
if state == "Start":
```

```
    state = "Near Goal"
```

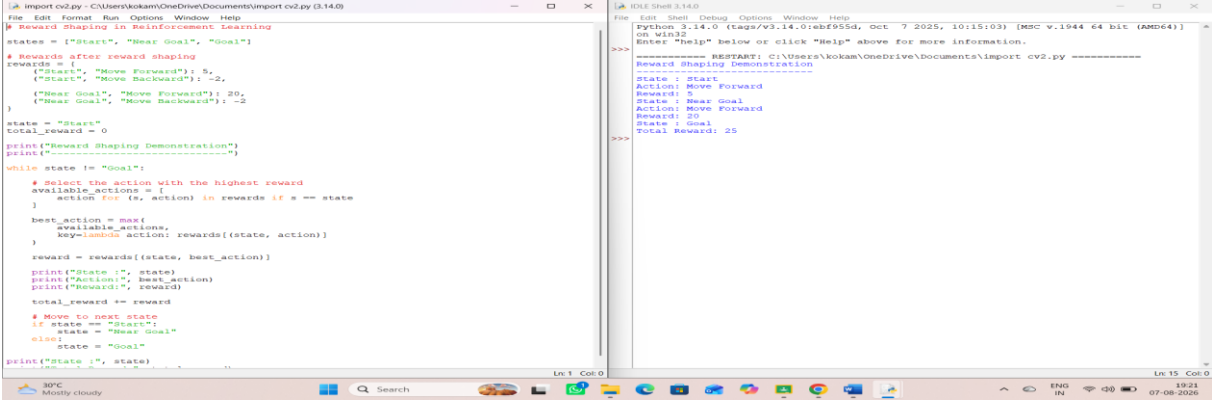
```
else:
```

```
    state = "Goal"
```

```
print("State :", state)
```

```
print("Total Reward:", total_reward)
```


Output;



The screenshot shows a Python IDE with two windows. The left window displays a Python script for a reinforcement learning task. The script defines states ('Start', 'Near Goal', 'Goal'), rewards, and actions ('Move Forward', 'Move Backward'). It uses a Q-learning algorithm to learn the optimal path from 'Start' to 'Goal'. The right window shows the output of the script, which prints the state, action, and reward at each step, and the total reward at the end.

```
import cv2.py - C:\Users\kokam\OneDrive\Documents\import cv2.py (3.14.0)
File Edit Format Run Options Window Help
# Reward Shaping in Reinforcement Learning
states = ["Start", "Near Goal", "Goal"]
# Rewards after reward shaping
rewards = {
    ("Start", "Move Forward"): 5,
    ("Start", "Move Backward"): -2,
    ("Near Goal", "Move Forward"): 20,
    ("Near Goal", "Move Backward"): -2
}
state = "Start"
total_reward = 0
print("Reward Shaping Demonstration")
print("-----")
while state != "Goal":
    # Select the action with the highest reward
    available_actions = [
        action for (s, action) in rewards if s == state
    ]
    best_action = max(
        available_actions,
        key=lambda action: rewards[(state, action)]
    )
    reward = rewards[(state, best_action)]
    print("State :", state)
    print("Action:", best_action)
    print("Reward:", reward)
    total_reward += reward
    # Move to next state
    if state == "Start":
        state = "Near Goal"
    else:
        state = "Goal"
    print("State :", state)
```

```
>>> ===== RESTART: C:\Users\kokam\OneDrive\Documents\import cv2.py =====
Reward Shaping Demonstration
--state--state-----
state: Start
Action: Move Forward
Reward: 5
State: Near Goal
Action: Move Forward
Reward: 20
State: Goal
Total Reward: 25
>>>
```

(5 marks)

Code of Conduct:

I K.Mounith(192425090) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: *K.Mounith*

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Technical Knowledge	25	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge

Problem Solving	25	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problemsolving ability; requires guidance	Unable to solve or apply concepts
Analytical Thinking	15	Analyzes questions critically and provides wellstructured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated
Communication	15	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively
Confidence	15	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism
Response to Questions	10	Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately